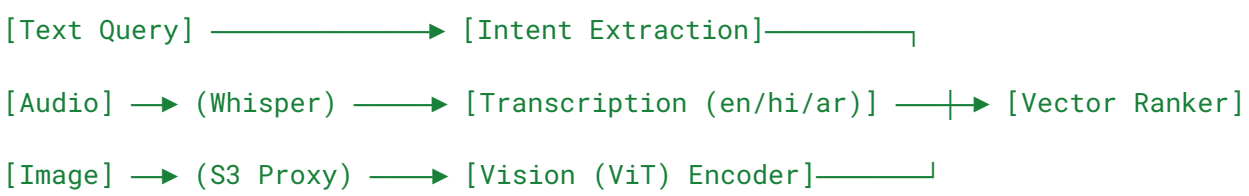
TECHNICAL ARCHITECTURE: MULTIMODAL COMMERCE CHAT SDK **Author:** Shreyansh

1. Client Integration & Networking Choices I designed the SDK as a decoupled JavaScript module targeting a sub-45KB gzipped footprint via Rollup. Because this will be injected into third-party retail sites, protecting their Core Web Vitals (LCP/FID) and isolating our UI via a Shadow DOM is the top priority.

For networking, I deliberately chose **Server-Sent Events (SSE)** over WebSockets for streaming the LLM chat responses. SSE is native to browsers, handles one-way token streaming perfectly, and avoids the operational overhead of managing stateful, persistent WebSocket connections at scale. Heavy media uploads (voice/images) bypass the stream and use standard HTTPS POST endpoints to prevent head-of-line blocking.

2. Identity & Multimodal Ingestion To provide historical recommendations, the SDK must resolve user identity. Upon initialization, the host site passes a hashed `user_id` or JWT into the SDK configuration. If the user is logged out, the SDK generates a local browser fingerprint to track anonymous clickstream data, which is merged upon future authentication.



- **Voice Ingestion:** The browser's `MediaRecorder` captures audio in WebM/Ogg. Instead of hardcoding language locales, the backend relies on OpenAI Whisper's auto-detection by omitting the language parameter (using `navigator.language` as a fallback hint) to dynamically transcribe English, Hindi, or Arabic into English text for downstream processing.
- **Shared Vector Space:** To compare image uploads and text queries mathematically, they must exist in the same latent space. I route both text queries and S3-uploaded images through the same multimodal embedding model (e.g., CLIP joint text/image encoders) to ensure the resulting vectors can be accurately compared.
- **Data Privacy (PII):** To comply with regional data regulations across jurisdictions, raw audio and image blobs are purged from the S3 proxy immediately upon vector encoding. All clickstream data is pseudonymized using the hashed `user_id`.

3. Retrieval & Recommendation Pipeline Real-time user intent must be combined with cold historical data. Doing this dynamically at runtime is a major latency bottleneck, so I split the pipeline:

- **Fast Retrieval:** The user's input vector is queried against a Milvus vector database, executing a standard Cosine Similarity search to pull a broad candidate pool of 50 visually or semantically similar products.
- **Contextual Re-Ranking:** I store the user's historical matrices (past brand purchases, frequently clicked categories) in a Redis Enterprise cluster, indexed by their `user_id`. An in-memory ranking worker intercepts the 50 raw Milvus candidates and applies a lightweight heuristic multiplier, boosting the score of items that align with the Redis profile.
- ***Trade-off**:* I chose a heuristic re-ranker over a deep-learning recommendation model (DLRM) to keep backend P99 latency under 100ms and minimize compute costs at scale.

4. Relevant Hands-On Experience

- **calai-ugc-engine**
  - *Relevance:* Proves my ability to build real-time, streaming AI interfaces. I engineered this full-stack Next.js engine using the Vercel AI SDK to stream dynamic content. I handled the exact edge cases required for the SDK, including isolated UI rendering, asynchronous asset pipelines, and automated `onError` CDN fallbacks for media stability.
- **pixii-aeo-diagnostic**
  - *Relevance:* Demonstrates my backend capability in handling semantic data and LLM routing. I architected this diagnostic platform using Python worker threads to execute parallel LLM orchestration (GPT-4o/Claude 3.5), directly mirroring the asynchronous parsing pipelines needed to transform unstructured text/voice into structured search parameters.